SQL START! 2025

Fondamenti di SQL Server: metodi di accesso ai dati

Gianluca Hotz

ghotz@ugiss.org

Sponsors & Organizers

Bio

- Gianluca Hotz | @glhotz | ghotz@ugiss.org
- Consulente Indipendente
  - 30 anni con SQL Server (dalla versione 4.21 nel 1996)
  - «Database modeling & development, sizing & administration, modernization (upgrades & migrations), performance tuning, security»
- YNAP SQL Server DBA Team (part-time)
- «Community»
  - 25+ anni Microsoft MVP SQL Server/Data Platform (dal 1998)
  - «VMware Experts SQL Server»
  - Fondatore e presidente UGISS (ex «PASS Chapter»)

Tabelle e Indici

Implementazione Tabelle

- Implementazione fisica
  - Tabelle «Disk Based»
  - Tabelle «Memory-Optimized» («In-Memory OLTP»)
- Implementazione funzionale
  - Tabelle «Temporal»
  - Tabelle «Edge» e «Node» («SQL Graph»)
  - Tabelle «Ledger»

Tabelle «Disk-based»

- Tabelle implementate tramite
  - Heap
  - B^+-Tree
  - Columnstore
- Partizionate

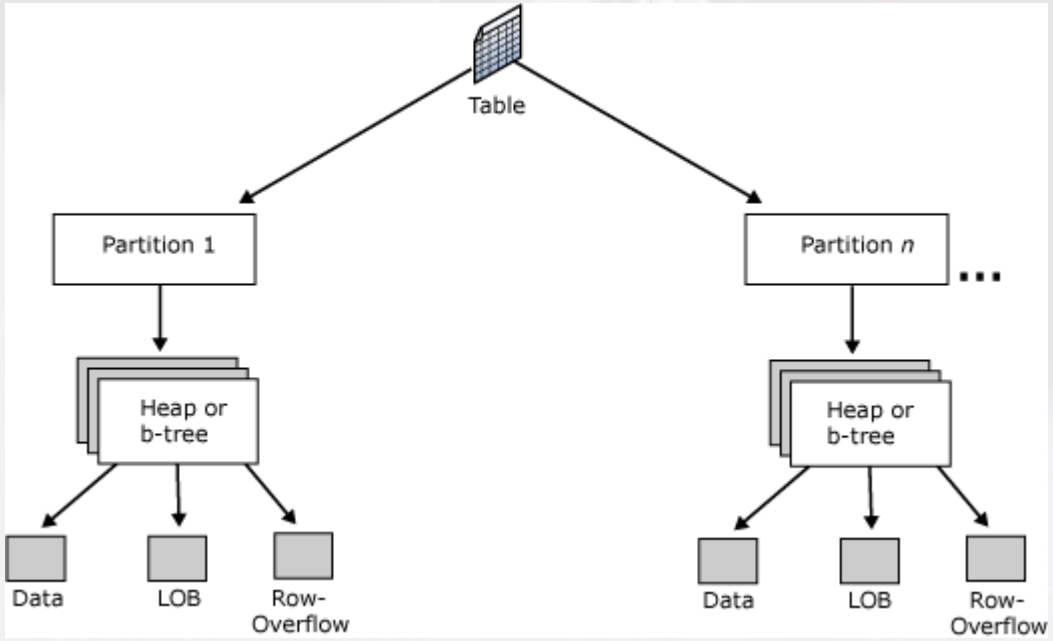
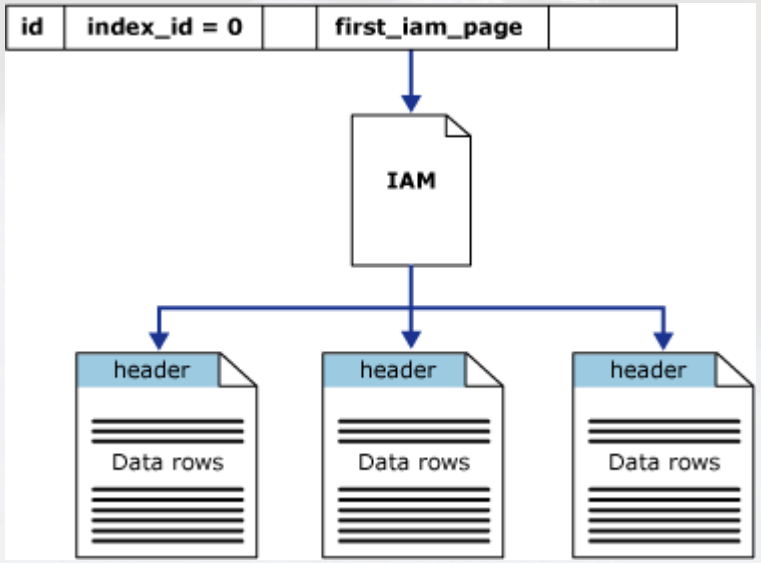


Tabella senza indici (HEAP)

- Tabella senza indice «Clustered»
  - Pagine localizzate da IAM
- Accesso ai dati
  - Solo «Table Scan»



Indici e accesso ai dati

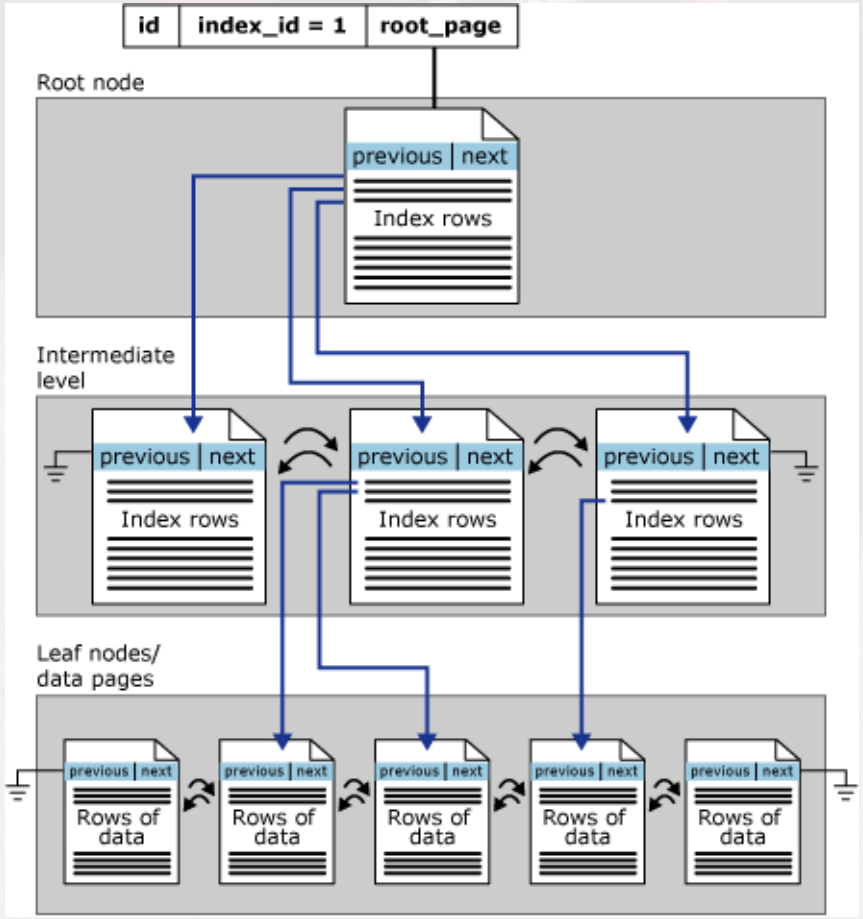
- Come accedere ai dati
  - effettuando una scansione di tutte le pagine dati
  - utilizzando un indice che punta alle pagine dati
- Perché creare un indice
  - velocizza l’accesso ai dati
  - imporre un vincolo di univocità
- Perché non creare un indice
  - consuma spazio disco
  - calo prestazioni per aggiornamenti

Tipologie di indici «Disk-based»

- Indici tradizionali
  - Si applicano a tabelle e viste
    - Indice «clustered» (primario)
    - Indici «non clustered» (secondari)
  - Altre caratteristiche
    - Univocità
    - Inclusione di colonne
    - Indici filtrati
- Indici speciali
  - «Full-text»
  - «XML»
  - Spaziali
  - «Columnstore»
  - «JSON» (2025+)

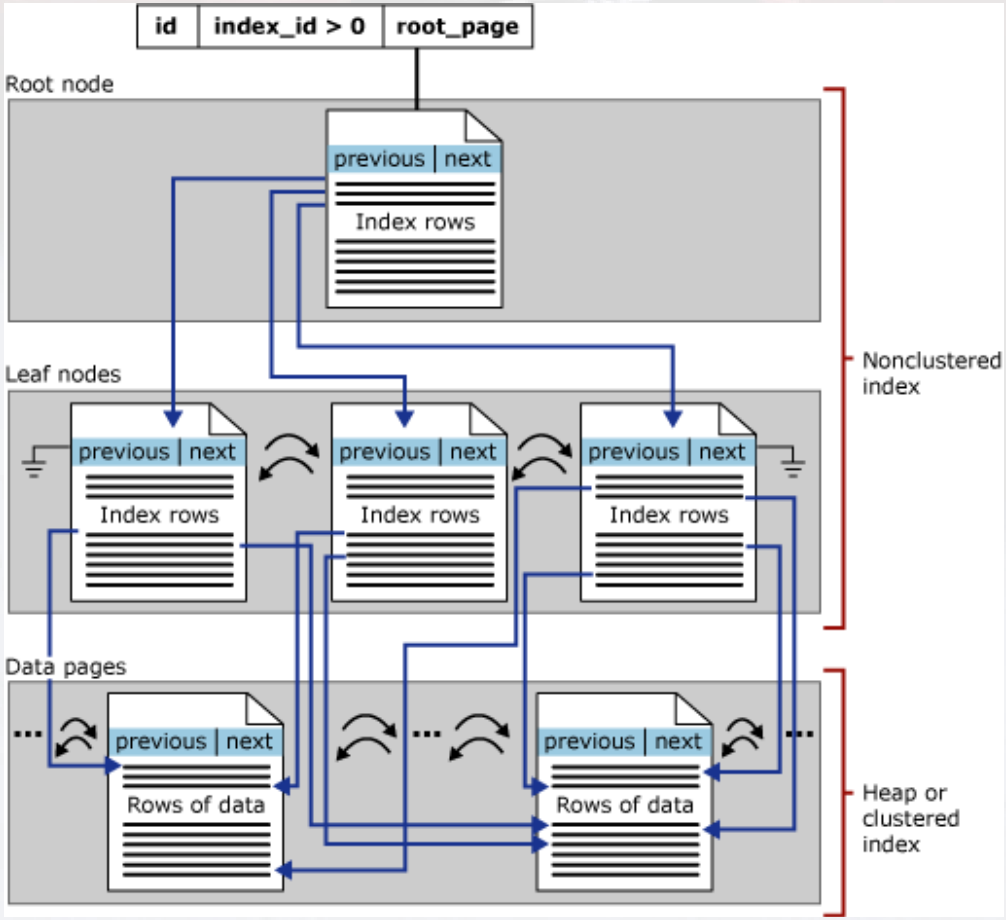
Tabelle con Indice «Clustered»

- B^+-Tree
  - B-Tree + liste doppia entrata
- Pagine foglia
  - sono le righe della tabella
- Metodi di accesso
  - «Index Scan» completo o parziale
  - «Index Seek»
- «Uniquifier»
  - 4 byte se indice non UNIQUE
  - Usato solo internamente



Indici «Non-Clustered»

- B^+-Tree
  - B-Tree + liste doppia entrata
- Pagine foglia
  - Colonne incluse
  - Puntatori a tabella
    - RID (fisico) se HEAP
    - Clustering Key (logico)
- Metodi di accesso
  - «Index Scan» completo o parziale
  - «Index Seek»



Colonne Incluse (non chiave)

- Clausola INCLUDE
- Colonne chiave
  - Limite 16 colonne, 900 byte
    - SQL Server 2016+ 1700 byte per indici «non-clustered»
  - Utili per ordinamento, filtri, raggruppamenti, join ecc.
- Colonne non chiave
  - Copertura Query (non servono «Lookups»)

Indici filtrati

- Predicato clausola WHERE
- Esempi
  - UNIQUE valori NOT NULL (altrimenti solo un valore!)
  - Colonne con molti valori NULL

Metodi di Accesso

«Allocation-order Scan»

- Utilizza pagine IAM
  - Letture in ordine per file
- Unica modalità per **«Table Scan»** di HEAP
- Modalità per scansione completa di B+-Tree
  - **«Unordered Clustered Index Scan»**
    - Scansione completa (tutte le righe) tabella con indice «Clustered»
    - «Unordered» perché ordinamento logico non richiesto (ORDER BY)
  - **«Unordered Covering Non-Clustered Index Scan»**
    - Scansione completa indice «Non-Clustered» di copertura
    - Tutte le pagine foglia soddisfano la richiesta senza ordinamento logico (ORDER BY)

Requisiti «Allocation-order Scan»

- Dimensione, almeno 64 pagine
- Condizioni che prevengono lettura righe modificate
  - livello di isolamento READ UNCOMMITTED (o NOLOCK/READUNCOMMITTED)
  - TABLOCK con isolamento non RCSI
  - TABLOCKX con isolamento RCSI (o TABLOCK e HINT per locking)
  - Database in sola lettura
  - Indice può acquisire solo PAGE lock (non è default)

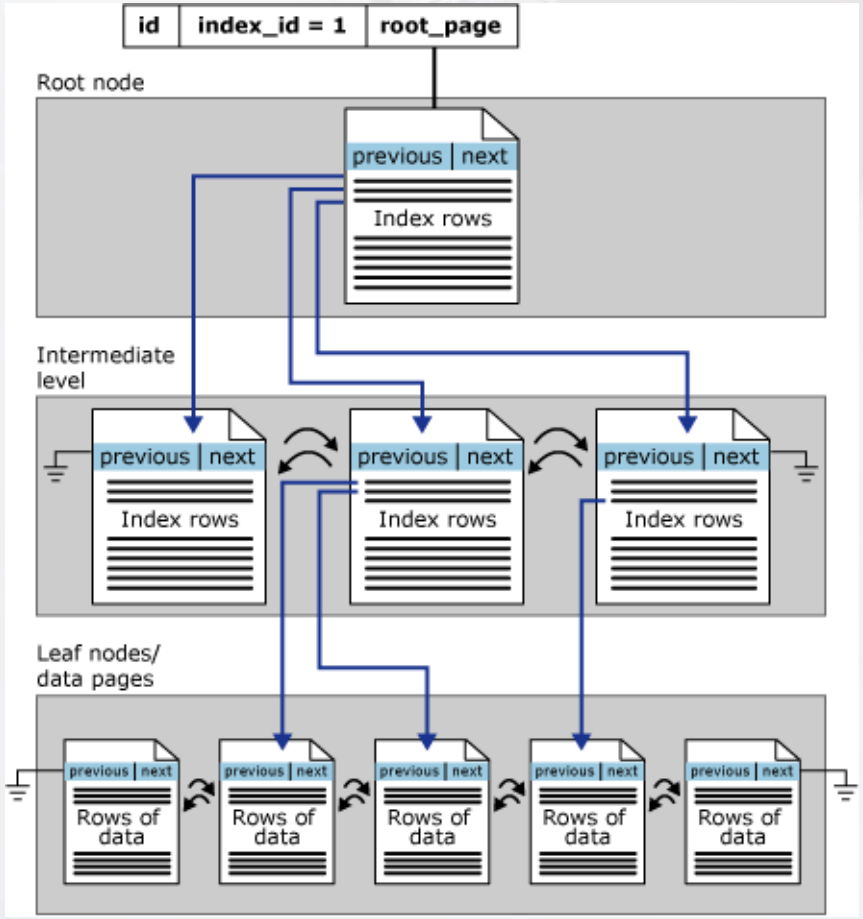
«Index-order Scan»

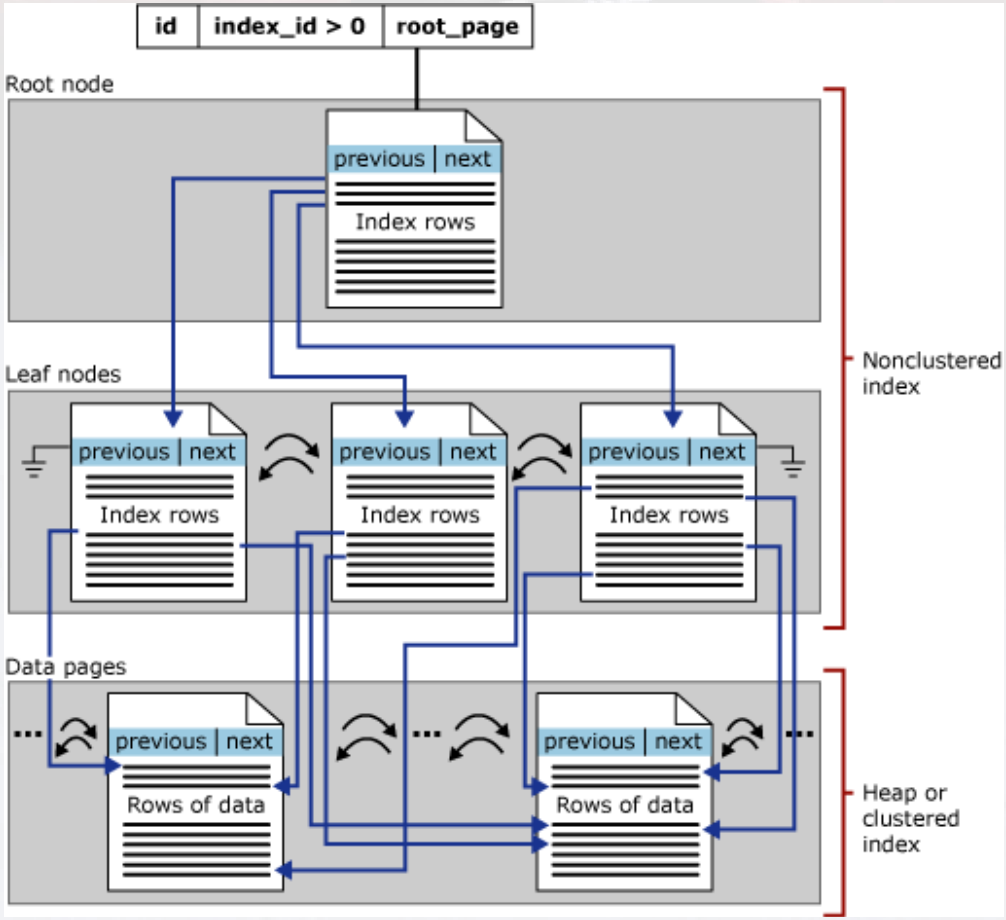
- Utilizza ordine dell’indice (pagine foglia e «Linked List»)
- Modalità alternativa per scansione completa di B+-Tree
  - **«Unordered Clustered Index Scan»**
  - **«Unordered Covering Non-Clustered Index Scan»**
  - **«Ordered Clustered Index Scan»**
  - **«Ordered Covering Non-Clustered Index Scan»**
- Risultato non necessariamente ordinato!
  - es. parallelismo, partizionamento, «Advanced Scan»
  - Clausola ORDER BY unica garanzia che sia ordinato

Indici discendenti

- Clausola DESC creazione indice
- «Storage Engine» può effettuare «Backward Scan»
  - Scansione in ordine inverso
  - Tramite «Linked List»
  - Parallelismo non ancora supportato! (solo «Forward Scan»)

Demo «Scan»


id
index_id = 1
root_page
Root node
previous | next
Index rows
Intermediate level
previous | next
Index rows
previous | next
Index rows
previous | next
Index rows
Leaf nodes/ data pages
previous | next
Rows of data
previous | next
Rows of data
previous | next
Rows of data
previous | next
Rows of data
previous | next
Rows of data


id
index_id > 0
root_page
Root node
previous | next
Index rows
Leaf nodes
previous | next
Index rows
previous | next
Index rows
previous | next
Index rows
Nonclustered index
Data pages
previous | next
Rows of data
previous | next
Rows of data
Heap or clustered index

«Read-Ahead»

- Meccanismo di «Pre-fetching»
- Anticipa letture pagine dati/indici
  - Sequenziale («allocation-order scan» o «index-order scan»
  - Random «Pre-fetching» (chiavi foglie indici «non-clustered»)
- Più efficienza da sovrapposizione computazione e accesso ai dati
- Block Size da 128KB a 512KB (8MB Columnstore)
- Outstanding da 2MB (Edizione Standard) a 8MB (Edizione Enterprise)

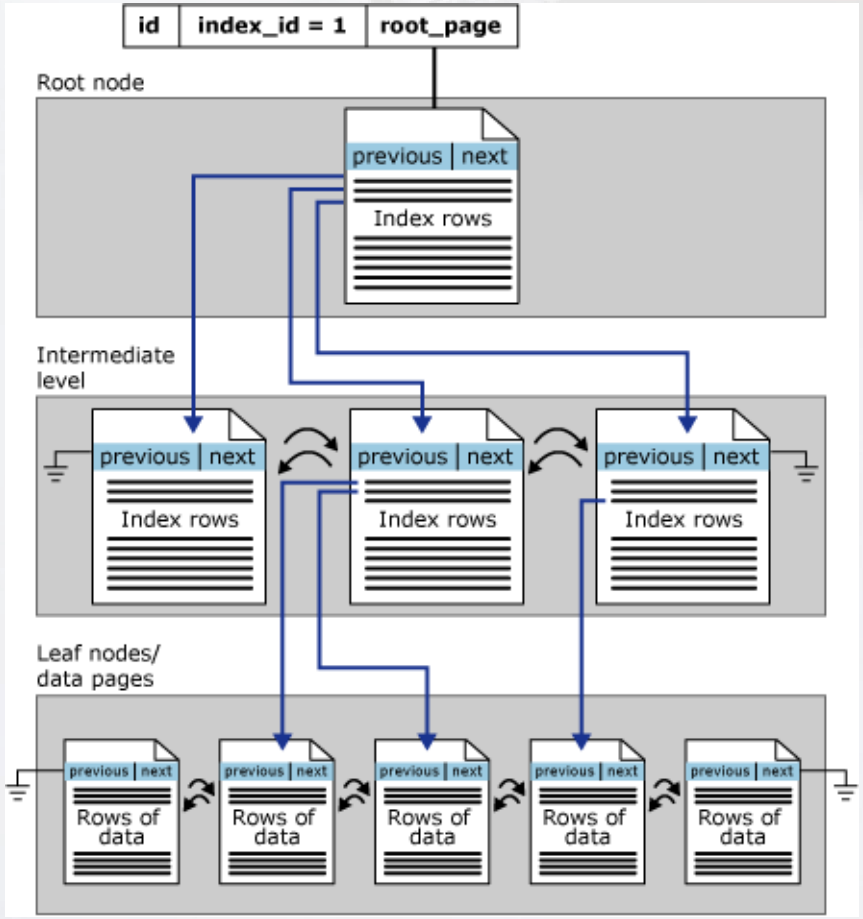
«Advanced Scanning»

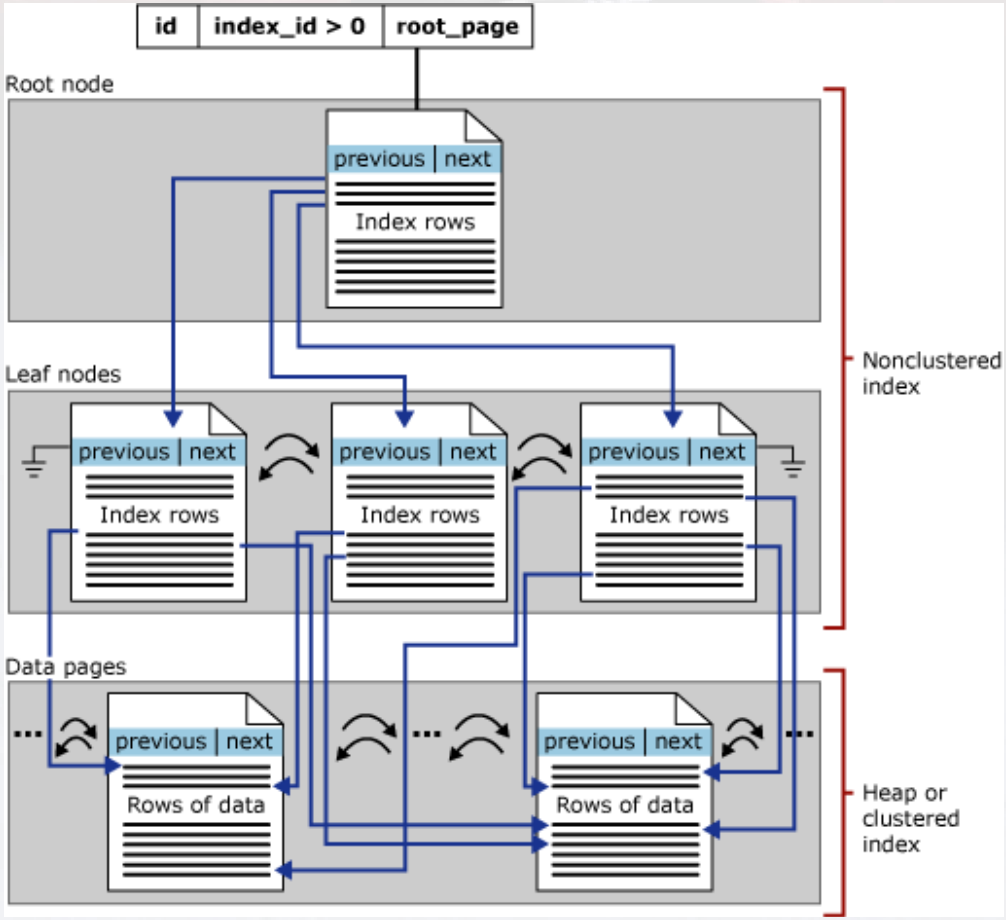
- Condivisione «Table Scan» solo per Enterprise Edition
- Meccanismo «Merry-go-round»
  - Tabella da 500.000 pagine
  - Sessione 1 comincia «Table Scan» da pagina 1
  - Arrivata a 100.000, sessione 2 inizia «Table Scan»
  - Pagine da 100.001 fornite a entrambe
  - Arrivate a 200.000 sessione 3 inizia «Table Scan»
  - Pagine da 200.001 fornite a tutte e tre
  - Arrivate a 500.000 sessione 1 termina, sessioni 2 e 3 continuano da pagina 1
  - Arrivate a 100.000 sessione 2 termina, sessione 3 continua fino a 200.000

Altre modalità di accesso

- «Non-Clustered Index Seek + Range Scan + Lookups»
- «Unordered Non-Clustered Index Scan + Lookups»
- «Clustered Index Seek + Range Scan»
- «Covering Non-Clustered Index Seek + Range Scan»

Demo altre modalità di accesso

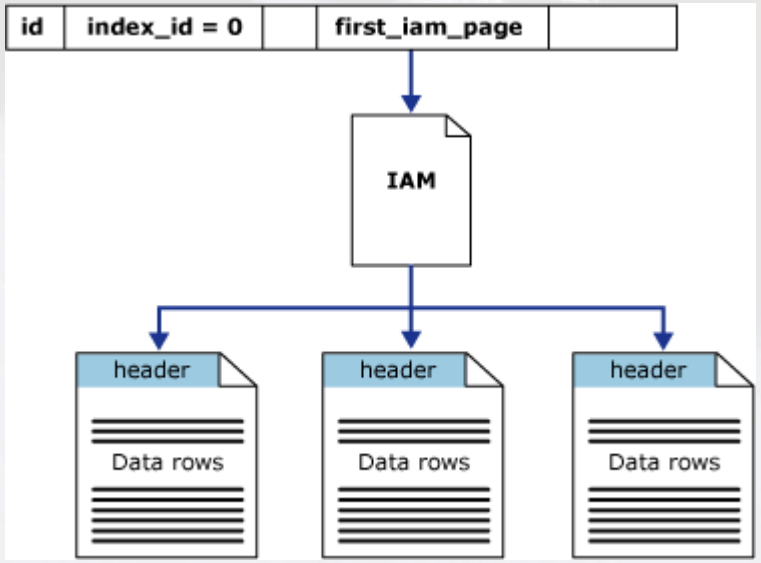




Modifiche ai dati

Aggiornamento HEAP

- «Forwarding Pointers»
  - A seguito Aggiornamento valori tipi dato a lunghezza varabile
  - Riga non può essere contenuta nella pagina originale
  - Puntatore alla nuova posizione
  - ALTER TABLE … REBUILD

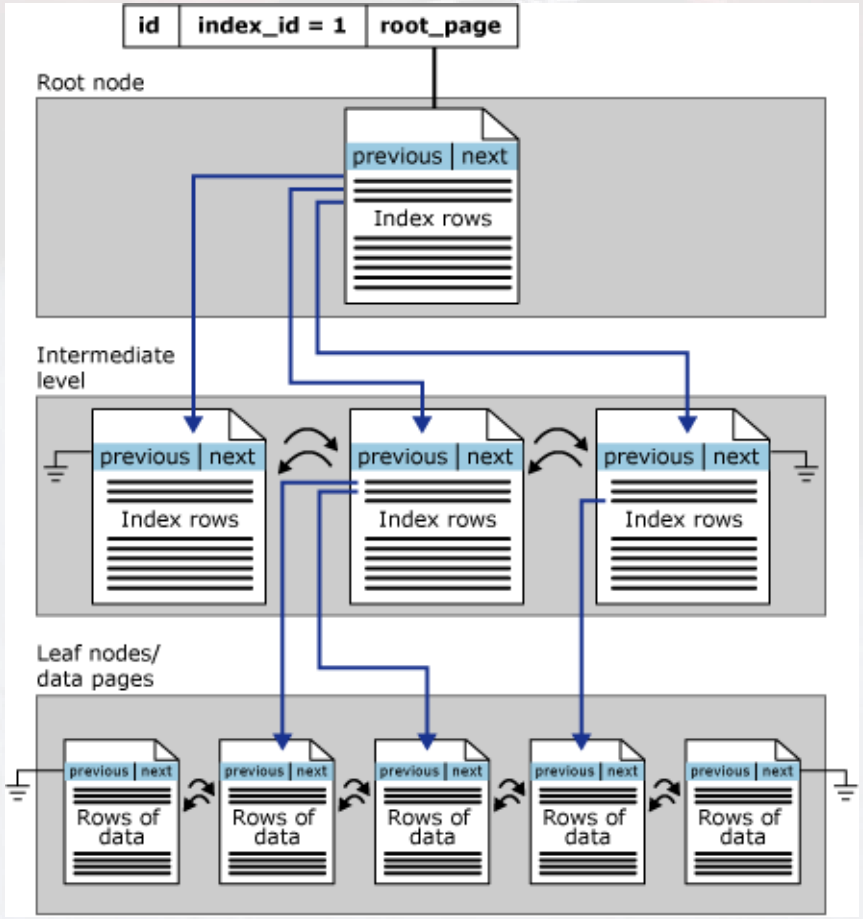


Demo

- «Forwarding Pointers»

«Page-Split» con B$^+$-Tree

- Inserimento/aggiornamento
  - Allocazione nuova pagina
  - 50% righe spostate nuova pagina
  - puntatori pagine aggiornati per mantenere ordinamento logico
- Valore crescente monotonico
  - “mezzo” page-split
  - allocazione nuova pagina senza spostamento righe
  - Aggiornamento puntatori



Frammentazione «Interna» B^+-Tree

- Spazio libero nelle pagine (bassa densità valori)
  - Consumo di spazio elevato, sono necessarie più pagine
- Desiderabile per applicazioni OLTP
  - Spazio libero per nuovi inserimenti (senza «Page-Split»)
  - Spazio libero per aggiornamenti (senza «Page-Split» o «Forwarding Pointer»)
- Non desiderabile per applicazioni OLAP
  - Necessarie più operazioni di I/O per «Table/Index Scan»
- Si controlla (raramente) con opzione CREATE/ALTER INDEX
  - FILLFACTOR = Percentuale riempimento pagine foglia (default=100=0)
  - PAD_INDEX = ON applica FILLFACTOR a pagine non-foglia
  - PAD_INDEX = OFF (default) lascia spazio per una chiave

Frammentazione «Logica» B^+-Tree

- Pagine indice «out-of-order»
  - Pagina punta logicamente a pagina non consecutiva fisicamente
  - Qualche volta chiamata frammentazione «Esterna»
- Per ridurla
  - ALTER INDEX … REBUILD
  - ALTER INDEX … REORG

Frammentazione «Extent» B^+-Tree

- «Extent out-of-order» di un HEAP
  - Pagina successiva non in «extent» corrente o consecutivo fisicamente
  - Anche questa qualche volta chiamata frammentazione «Esterna»
- Per ridurla
  - CREATE/DROP Indice Clustered
  - ALTER TABLE .. REBUILD rimuove solo «Forwarding Pointer»

Demo

- Frammentazione